



ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ
Мэдээлэл, Холбооны Технологийн Сургууль

F.CSM101 Программчлалын үндсэн аргууд

Өгөгдлийн хийсвэрлэл & Объект хандалгат парадигм

Лекц 9

Доктор (Ph.D) Ганбаатарын ГАНБАТ

2023-2024 Хавар

- Хэсэг I. Өгөгдлийн хийсвэр
 - Өгөгдлийн хийсвэр төрөл (ADT)
 - Мэдээллийн далдлалт
 - Модуль
- Хэсэг II. Объект хандалгат парадигм
 - ADT-ийн боломж
 - Онолын суурь



Физик машин нь зөвхөн нэгэн төрлийн битүүд дээр ажилладаг.
**Дээд түвшний хэлний төрлүүд нь
энэхүү үл хуваагдах цул байдалд
бүтэцлэлт оруулдаг.**

(Утга тус бүрийг багцалт (wrap) хэлбэрээр авч үздэг)

- Утга нь үйлдлийнхээ хамт инкапсуляц (encapsulations) дотор багцлагддаг.
- **Төрлийн аюулгүй хэлний** хувьд инкапсуляц нь өгөгдлийн дүрслэлдээ хандахыг зөвшөөрдөггүй (бүрэн битүүмжлэлтэй).
- Нийлмэл төрөл нь хэлээр predefined байдаг, явцуу хүрээнд хэрэглэгддэг
- Шинэ төрөл тодорхойлолт өгөгдлийн хийсвэрлэлийн цөөн хэдэн аргатай.

Өгөгдлийн хийсвэр төрөл, Өгөгдлийн далдлалт, Модуль

➤ Өгөгдлийн хийсвэр төрөл (Abstract data types - ADT)

PL-ний төрлөөр шинэ төрлийг авч үзэхэд predefined төрлийн ашиглалт шиг хийсвэрлэл байгуулах боломжгүй

```
type Int_Stack = struct{  
    int P[100]; // the stack proper  
    int top;    // first readable element  
}
```

```
Int_Stack create_stack(){  
    Int_Stack s = new Int_Stack();  
    s.top = 0;  
    return s;  
}
```

```
Int_Stack push(Int_Stack s, int k){
```

```
int top(Int_Stack s){
```

```
Int_Stack pop(Int_Stack s){
```

```
bool empty(Int_Stack s){
```

- Нэрийн хатуу төрөл эквиваленттай ч **массив шиг хандахад саадгүй**.

```
int top(Int_Stack s){  
    return s.P[s.top];  
}
```

```
int second_from_top()(Int_Stack c){  
    return c.P[s.top - 1];  
}
```

- Ерөнхийдөө хэл нь хэрэгжилтийг далдалдаг predefined өгөгдлийн хийсвэрлэл (төрөл)-тэй ч программист шинээр зохиож чадахгүй.
- Заримс PL-д дараахь байдлаар ADT тодорхойлогддог.
 1. Төрлийн нэр.
 2. Төрөлд зориулсан хэрэгжилт (эсвэл дүрслэл).
 3. Төрлийн утгыг өөрчлөх үйлдлийн нэрс, тэдгээрийн төрлүүдийн хамт.
 4. Үйлдэл бүрийн хувьд 2-т заасан дүрслэлийг ашигласан хэрэгжилт.
 5. Капсулын аюулгүйн хамгаалалт нь нэр, үйлдлийг хэрэгжилтээс тусгаарладаг.
- ADT нь бүрэн битүүмжтэй капсул:
 - Concrete Type, Signature, Operations
 - Хэнд ч нээлттэй гадарга дээрх шинэ төрлийн болон үйлдлийн нэр ба төрөл.
 - Дотор нь гаднаа үл үзэгдэх төрөл, түүний үйлдлийн хэрэгжилтүүдтэй.

ADT >> Signature

- Капсул гадаргууг **signature** буюу **интерфейс** гэнэ.
- Дотоод руу хандахыг капсулаар хянагддаг
 - Мэдээллийн **тогтвортой байдлыг** хангадаг.
- Дотор тал нь түүний хэрэгжилт (implementation)

```
abstype Int_Stack{  
    type Int_Stack = struct{  
        int P[100];  
        int n;  
        int top;  
    }  
    signature  
        Int_Stack create_stack();  
        Int_Stack push(Int_Stack s, int k);  
        int top(Int_Stack s);  
        Int_Stack pop(Int_Stack s);  
        bool empty(Int_Stack s);  
    operations  
        Int_Stack create_stack(){  
        Int_Stack push(Int_Stack s, int k){  
        int top(Int_Stack s){  
        Int_Stack pop(Int_Stack s){  
        bool empty(Int_Stack s){  
        }
```

Мэдээлэл далдлалт

- Интерфейс болон хэрэгжилтийн зааг нь ПХ хөгжүүлэлтэд маш чухал
 - Компонентийн хэрэглээг тодорхойлолтоос нь салгах боломж.
 - *Функц нь их биеээ (хэрэгжилтийг) хийсвэрлэн (ө.х далдалдаг) – Нэр, парам тоо, төрлөөс бүрдсэн интерфейсийг (ө.х signature) харуулдаг.*
 - Үйлдэл хэрэгжиж байгаа нь, өгөгдлийн дүрслэл нь далд **(Мэдээллийн далдлалт)**: Хийсвэрлэлийг зөрчиж болохгүй гэдгийг баталгаажуулж чадна.
- Мэдээлэл далдлалтын хамгийн сонирхолтой нь:
 - Тодорхой нөхцлийн дагуу интерфэйсийг ижил байлгаж, нэг ADT хэрэгжилтийг ийн өөр нэгээр орлуулах боломжтой.
- Int_Stack-ийн анхны бүтцийг тодорхой **төсөөллийн** дагуу сольвол программд **ажиглагдахуйц (observable)** нөлөө үзүүлэх ёсгүй.
 - Интерфэйс хэрэглэгч үйлдлээс юу хүлээж байгаад **төсөөлөл** нь төвлөрдөг.

МЭДЭЭЛЭЛ ДАЛДЛАЛТ >> Зааварчилгаа (Specification)

- ADT-ийн үйлдлийн *семантик* тайлбар нь бодит төрлөөр бус харин ерөнхий хийсвэр харьцаан суурилсан **зааварчилгаа** байна. Int_Stack:
 - create_stack нь хоосон стек үүсгэдэг.
 - push нь стек рүү элемент оруулдаг.
 - top нь стекийг өөрөө өөрчлөхгүйгээр стекийн дээд талд байгаа элементийг буцаана. Стек хоосон байж болохгүй.
 - pop нь стекээс дээд элемент устгадаг. Стек хоосон байж болохгүй.
 - empty нь зөвхөн стек хоосон байвал үнэн байна.
- **Дүрслэлийн бие даасан байдлын зарчим (principles of Representation Independence):** Уг зааварчилгааны дагуу Int_Stack клиент нь олон тодорхойлолтын ялгааг олж харахгүй.

Дүрслэлийн бие даасан байдал (Representation independence)

ADT-ийн нэг зааварчилгааны хоёр зөв хэрэгжилтийг тухайн төрөл нь хэрэглэгч дээр харагдахуйц ялгарахгүй байх.

- Хэрэглэгч дээр ямар нэгэн (шинэ) алдаа гарахгүй байхаар ижил төстэй (жнь, илүү үр ашигтай) хувилбараар хэрэгжилтийг солих боломжтой.
- Ижил зааварчилгааны хувьд хоёулангийнх нь хэрэгжилт зөв байх нь баталгаатай байх ёстой.
- Сул хувилбар байдаг бөгөөд энэ нь зөвхөн signature-ын хувьд зөв байх.
 - Хийсвэр өгөгдлийн төрөлтэй, төрлийн аюулгүй хэлэнд нэг ADT-г нөгөө signature-тэй (гэхдээ өөр хэрэгжилттэй) солих нь төрлийн алдаа үүсгэхгүй.
 - Бидний авч үзсэн төсөөллийн дагуу (төрлийн аюулгүй байдал ба ADT) төрлийн үнэлгээгээр нотлогдож болох теорем юм. ML болон CLU зэрэг хэлүүд энэ хэлбэрийг агуулдаг.

operations

```
Int_Stack create_stack(){
    Int_Stack s = new Int_Stack();
    s.n = 0;
    s.top = 0;
    return s;
}
Int_Stack push(Int_Stack s, int k){
    if (s.n == 100) error;
    s.n = s.n + 1;
    s.P[s.top] = k;
    s.top = s.top + 1;
    return s;
}
int top(Int_Stack s){
    return s.P[s.top];
}
Int_Stack pop(Int_Stack s){
    if (s.n == 0) error;
    s.n = s.n - 1;
    s.top = s.top - 1;
    return s;
}
bool empty(Int_Stack s){
    return (s.n == 0);
}
```

operations

```
Int_Stack create_stack(){
    return null;
}
Int_Stack push(Int_Stack s, int k){
    Int_Stack tmp = new Int_Stack(); // new element
    tmp.info = k;
    tmp.next = s; // chain on
    return tmp;
}
int top(Int_Stack s){
    return s.info;
}
Int_Stack pop(Int_Stack s){
    return s.next;
}
bool empty(Int_Stack s){
    return (s == null);
}
```

- Хийсвэрлэл нь харилцан хамааралтай хэд хэдэн төрлөөс (өгөгдлийн бүтэц) бүрдэх нь түгээмэл байдаг
 - Хязгаарлагдмал (ө.х хийсвэр) байдлаар (бүх үйлдэл харагдахгүй, хэрэгжилтийг далдалсан) харуулдаг инкапсуляцыг хэрэгжүүлэх хэл шинжлэлийн механизм нь **модуль/packages** юм.
 - Программыг статик байдлаар дэд хэсгүүдэд хуваах боломжтой. Тус бүртээ өгөгдөл (төрөл, хувьсагч), үйлдлүүд (функц, код) -той.
 - Өгөгдөл/функцүүдийг нэгтгэж, харагдах байдлын дүрмийг тодорхойлох байдлаар инкапсуляц, мэдээллийг далдлалтыг бүрдүүлдэг.
- Семантик талаасаа модуль ба ADT нь их ялгаа байхгүй.
- Модулиуд ихэвчлэн тусдаа компиляцтай байдаг.
- link-time задаргааг шаарддагаараа параметр полиморфизмын зарим хэлбэртэй холбоотой.

МОДУЛЬ ➤ Бүтэц, зохион байгуулалт

```
module Buffer imports Counter{
  public
    type Buf;
    void insert(reference Buf f, int n);
    int get(Buf b);
    Count c; // how many times buffer has been used
  private imports Queue{
    type Buf = Queue;
    void insert(reference Buf b, int n){
      inqueue(b,n);
      inc(c);
    }
    int get(Buf b){
      return dequeue(b);
      inc(c);
    }
    init_counter(c);    // module initialisation part
  }
}
```

```
module Counter{
  public
    type Count;
    void init_counter(reference Count c);
    int get(Count c);
    void inc(reference Count c);
  private
    type Count = int;
    void init_counter(reference Count c){
      c=0;
    }
    int get(Count c){
      return c;
    }
    void inc(reference Count c){
      c = c+1;
    }
  }
  module Queue{
    public
      type Queue;
      inqueue(reference Queue q, int n);
      int dequeue(reference Queue q);
      ...
    private
      void bookkeep(reference Queue q){
        ...
      }
      ...
  }
}
```

ADT нь инкапсуляц, мэдээлэл далдлалттай боловч комплекс дизайнд хэрэглэхэд уян хатан биш.

Бидэнд шаардлагатай төрөл

- ***Инкапсуляц, мэдээлэл далдлалттай (ADT-д байдаг шиг)***
- ***Өгөгдсөн тодорхойлолтоос тусгай нөхцлийн дор үйлдлүүд хэрэгжүүлэх удамшлыг дэмждэг.***
- ***Тусгай бүтцэд зориулсан үйлдлүүдийн хүрээнд тодорхойлсон нийцтэй байдлын тухай ойлголтоор бүрдүүлсэн.***
- ***Хэрэглэж буй аргументуудын “үр ашигтай төрлөөс” хамааран үйлдлүүдийг динамик сонгох боломжтой.***

Counter төрлийн бодит дүрслэл нь int.
тоолуурыг тэглэх, унших болон утгыг
нэмэгдүүлэх үйлдэлтэй.

+ reset дуудалтыг тоолох үйлдэл нэмэх?

- ADT-ын инкапсуляц ба мэдээлэл дадлалт цэвэрхэн, үр ашигтай.
 - Ялангуяа data, методыг нэг бүтцэд хавсаргадаг онцлогтой.
- Илэрхий уян хатан бус байдлын зардлаар олж байгаа чухал зарчим
 - Хийсвэрлэлийг өргөжүүлэх эсвэл дахин ашиглах үед шаардагддаг.

```
abstype Counter{  
  type Counter = int;  
  signature  
    void reset(Counter x);  
    int get(Counter x);  
    void inc(Counter x);  
  operations  
    void reset(Counter x){  
      x = 0;  
    }  
    int get(Counter c){  
      return x;  
    }  
    void inc(Counter c){  
      x = x+1;  
    }  
}
```



```
abstype NewCounter1{
  type NewCounter1 = struct{
    int c;
    int num_reset = 0;
  }

  signature
    void reset(NewCounter1 x);
    int get(NewCounter1 x);
    void inc(NewCounter1 x);
    int howmany_resets(NewCounter1 x);
  operations
    void reset(NewCounter1 x){
      x.c = 0;
      x.num_reset = x.num_reset+1;
    }
    int get(NewCounter1 x){
      return x.c;
    }
    void inc(NewCounter1 x){
      x.c = x.c+1;
    }
    int howmany_resets(NewCounter1 x){
      return x.num_reset;
    }
}
```

Өмнөхтэй ижил байдлаар бүрэн шинэ ADT тодорхойлж болно.

- Инкапсуляц - ОК
- Энгийн тоолуурын үйлдлүүд давтсан
 - ижил нэртэй ч аргументын ялгаатай төрөлтэй үйлдлүүд (дахин тодорхойлолт)
 - Сөрөг нөлөө нь ноцтой, (ядаргаа)
- Өмнөх класстай ямар ч хамаарал байхгүй (reuse)
 - Жнь: алдаа засах?


```
abstype NewCounter2{
  type NewCounter2 = struct{
    Counter c;
    int num_reset = 0;
  }

  signature
    void reset(NewCounter2 x);
    int get(NewCounter2 x);
    void inc(NewCounter2 x);
    int howmany_resets(NewCounter2 x);
  operations
    void reset(NewCounter2 x){
      reset(x.c);
      x.num_reset = x.num_reset+1;
    }
    int get(NewCounter2 x){
      return get(x.c);
    }
    void inc(NewCounter2 x){
      inc(x.c);
    }
    int howmany_resets(NewCounter2 x){
      return x.num_reset;
    }
}
```

Counter ADT хэрэглэн өргөтгөсөн тоолуур тодорхойлох.

- Шийдэл нь өмнөхөөс илүү
- NewCounter2 өөрчлөгдөхгүй үйлдлүүд ч байна (get, inc - inherit)
- Массив: Counter + NewCounter2

```
Counter V[100];
for (int i=1; i<100; i=i+1)
  reset(V[i]);
```

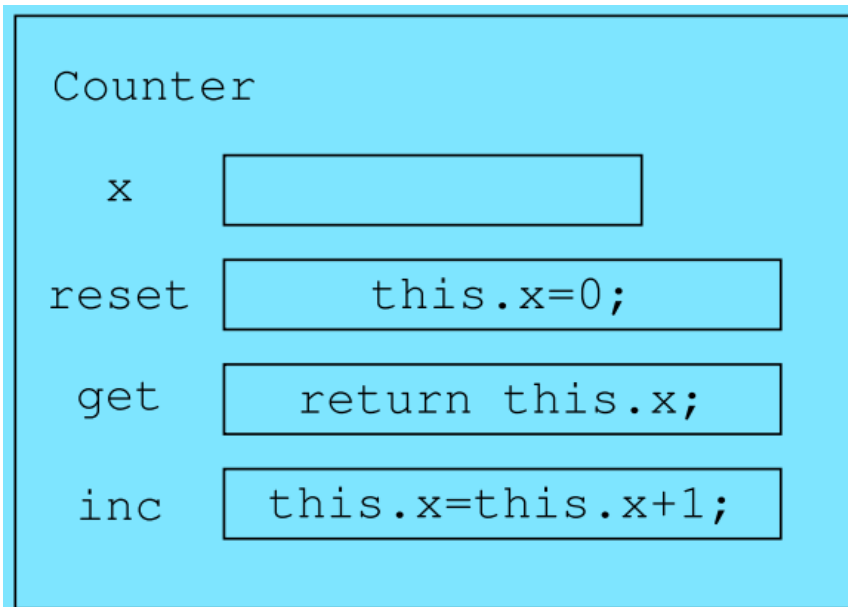
- Төрлийн нийцтэй байдал - OK
- Counter-ын reset ажиллана.
- NewCounter2-ын төлвийн хувьд инкапсуляц эвдэрч байна (num_reset)

#>> Объект хандалгат парадигм

- Объект хандалтат парадигмыг зохион байгуулах үндсэн ухагдахуун.
 1. Инкапсуляц болон хийсвэрлэл.
 2. Дэд төрөл, ө.х объектын функциональ байдал дээр суурилсан нийцтэй байдлын харьцаа.
 3. Удамшил, ө.х өөр объект эсвэл өөр класст өмнө нь тодорхойлсон методын хэрэгжилтийг дахин ашиглах боломж.
 4. Методын динамик сонголт.
- Эдгээр нь бусад парадигмд тус тусдаа байдаг ялгаатай механизмууд
- ПХ-ийн дизайн, түүнчлэн том системийн хувьд **ОХ парадигм**-ыг маш **сонирхолтой** болгодог зүйл бол **эдгээрийн харилцан үйлчлэл**

ОХ ПАРАДИГМ ➤ Объект

- ОХ хэлний гол бүтэц – өгөгдөл, тэдгээр дээрх үйлдлүүд агуулсан капсул
 - Энэ капсул нь объектын хандалт буюу гадна талын интерфейстэй.
 - Өгөгдөл нь үйлдлүүдтэйгээ "хамтдаа байх" гэдгээр ADT-тэй адил.
- Гэвч ADT-ээс зарчмын том ялгаатай.
 - ADT тай **хувьсагч** нь зөвхөн үйлдлүүдээрээ удирдах **өгөгдлийг** төлөөлдөг.
 - **Объект** бүр нь **өгөгдлийг** болон **үйлдлийг** инкапсуляц хийсэн контейнер

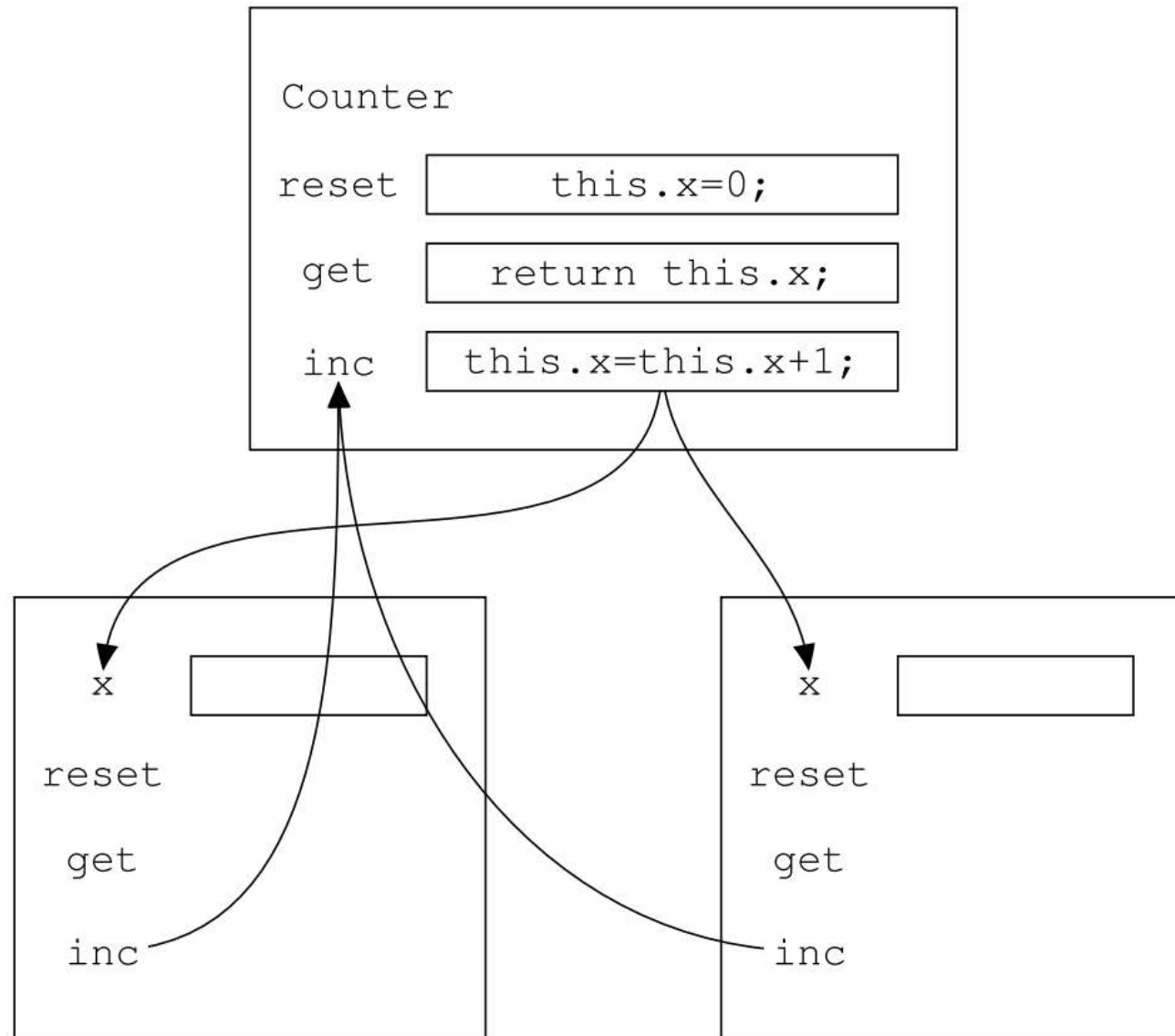


- Өөрчлөгдөх боломжтой өгөгдөл: x
 - *data members* эсвэл *instance attributes*
- Өгөгдлийг өөрчлөх үйлдлүүд: reset, get, inc
 - *Method (member function)*, объект дахь өгөгдөлд хандах боломжтой
 - Биелэлт нь metodyн нэр&парам-ыг *мессеж* болгон илгээх объект руу байдлаар хийгддэг¹⁸

- `object.method(parameters)`: object дээрх method-ын (parameters параметруудтэй) дуудалт
- Хүлээн авагч объект дотор кодын биелэлтийг ажиллуулна (this)
 - Метод дуудалт биелэгдэх методын **динамик сонголт**-той агуулдаг
 - Дата гишүүний хувьд адилхан зарчимтай, Дата нэгж нь статик бол Python, классгүй хэлүүд онцгой тохиолдол болно.
- **Капсулын битүүмжлэлийн түвшинг** объект өөрөө үүсээд тодорхойлно
 - Зарим үйлдлүүд хаана ч, эсвэл зарим өөр объектод харагдах боломжтой
 - Бүрэн хувийн байж болно, ө.х зөвхөн тухайн объект дотор
- **Зохион байгуулалтын механизм** шаардлагатай (Класс)
 - Ижил/төстэй бүтэцтэй объектуудыг бүлэглэх боломжийг олгодог.
 - Хэдийгээр объект бүр өөрийн гэсэн өгөгдөл, методыг агуулсан байхыг зарчмын хувьд зөвшөөрдөг ч энэ нь асар их хог хаягдлыг бий болгоно.
 - Хэрэгжилт талаасаа объект болгонд хуулбар үүснэ

- Класс нь объект олонлогийн загвар.
- Класстай хэлэнд объект динамик үүсгэгддэг.
 - Simula: класс нь локал хувьсагч, функц тодорхойлолт агуулсан идэвхжлийн бичлэгт заагч буцаадаг процедур (классын объектууд нь код блок (closures))
 - Smalltalk: класс нь хэл шинжлэл талаасаа, объект олонлогийн хэрэгжилтийн тодорхойлолтын схем объект.
 - C++, Java, Python: класс нь төрөл, А классын объектууд нь А төрлийн утгууд болно.
- Counter d = new Counter();
 - Үүсгэх (СО-н хуваарилалт, new)
 - Эхлүүлэх (класс-нэр, хаалт бүхий байгуулагч)

```
class Counter{  
    private int x;  
    public void reset(){  
        x = 0;  
    }  
    public int get(){  
        return x;  
    }  
    public void inc(){  
        x = x+1;  
    }  
}
```





ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ
Мэдээлэл, Холбооны Технологийн Сургууль

АНХААРАЛ ТАВЬСАНД БАЯРЛАЛАА